AKTE W3·1

Vernams Lochstreifen

KERN LÖSUNG

ARCHIVFUND

Die Archivarin hat den Lochstreifen von 1917 entschlüsselt. Doch wie funktioniert die Maschine, die Nachricht und Schlüssel Loch für Loch verknüpft – und warum stand auf dem Schlüssel „nach Gebrauch vernichten“? In dieser Akte lernst du die Sprache der Maschinen: **Bits**. Du baust das einzige Verfahren, das nachweislich nicht zu knacken ist – und erfährst, wie es trotzdem fällt.

Hilfsmittel: ASCII-Tabelle der Abitur-Anlage. Bits schreiben wir in Vierergruppen, das höchstwertige Bit links; XOR-Zeichen ⊕.

ARCHIVFUND · DER LOCHSTREIFEN VON 1917

Zwei Streifen aus einer Blechdose: acht Spuren nebeneinander, **Loch = 1**, kein Loch = 0. Jede Zeile ist ein Zeichen.

Geheimtext-Streifen

	128	64	32	16	8	4	2	1
1	●	●	●	●	·	●	·	·
2	·	●	●	·	●	●	·	·
3	●	·	·	·	●	·	●	●
4	·	·	●	·	·	●	·	●

Schlüssel-Streifen „nach Gebrauch vernichten!“

	128	64	32	16	8	4	2	1
1	●	·	●	●	·	·	●	·
2	·	·	●	●	●	·	·	●
3	●	●	·	·	·	●	·	●
4	·	●	●	·	●	●	●	·

Vergleiche die Streifen **Spalte für Spalte**. Finde eine Regel, die aus Geheimtext und Schlüssel ein sinnvolles Zeichen macht (Tipp: Großbuchstaben beginnen mit `010`).

Zeichen	Geheimtext	Schlüssel	Ergebnis der Regel	ASCII-Wert	Zeichen
1	`1111 0100`	`1011 0010`	`0100 0110`	70	**F**
2	`0110 1100`	`0011 1001`	`0101 0101`	85	**U**
3	`1000 1011`	`1100 0101`	`0100 1110`	78	**N**
4	`0010 0101`	`0110 1110`	`0100 1011`	75	**K**

Regel: **gleich → 0, verschieden → 1** (XOR). Nachricht: **FUNK**. Die „zwei Kerben“ auf dem Schlüsselstreifen führen zum Funkraum „Der doppelte Schlüssel“.

1 AUFTRAG ★ Zeichen werden Bits EINZELARBEIT

a) **Bestimme** mit der ASCII-Tabelle der Anlage und den Stellenwerten 128, 64, …, 1 die ASCII-Werte und Bits von B, O, J, E. Kontrolliere danach mit Archivwissen 1.

b) **Bestimme** umgekehrt das Zeichen zu `0101 0111` und zu `0100 1000`.

Zeichen	B	O	J	E	W	H
ASCII	66	79	74	69	87	72
Bits	`0100 0010`	`0100 1111`	`0100 1010`	`0100 0101`	`0101 0111`	`0100 1000`

© Frank Juchim · Kryptos · für den Unterricht

LÖSUNG P1

B 66 `0100 0010`, O 79 `0100 1111`, J 74 `0100 1010`, E 69 `0100 0101`; `0101 0111` = 64 + 16 + 4 + 2 + 1 = 87 = **W**, `0100 1000` = 64 + 8 = 72 = **H**.

Typischer Fehler: A = 1 statt 65 (P1).

ARCHIVWISSEN 1 · ZEICHEN ALS BITS

Ein **Bit** hat den Wert 0 oder 1. Acht Bits (ein **Byte**) stellen eine Zahl von 0 bis 255 als Dualzahl dar; die Stellen haben von links die Werte 128, 64, 32, 16, 8, 4, 2, 1. Der **ASCII-Code** (Anlage) ordnet jedem Zeichen eine Zahl zu: A = 65, B = 66, …, Z = 90, Leerzeichen = 32.

Beispiel: B = 66 = 64 + 2 = `0100 0010`. Wir schreiben Bits in **Vierergruppen**, das höchstwertige Bit links.

Zeichen	ASCII	128	64	32	16	8	4	2	1	Bits
B	66	0	1	0	0	0	0	1	0	`0100 0010`
O	79	0	1	0	0	1	1	1	1	`0100 1111`
J	74	0	1	0	0	1	0	1	0	`0100 1010`
E	69	0	1	0	0	0	1	0	1	`0100 0101`

Merksatz: *Großbuchstaben beginnen mit* `010`, *die restlichen 5 Bits sind die Nummer im Alphabet* (O = 15 → `010 01111`). Die vollständige ASCII-Tabelle steht in der Anlage.

ACHTUNG FALLE P1 · Drei Zählweisen

Der **ASCII-Wert** von A ist **65** – nicht 1 (Alphabetleiste der Anlage, A = 1 … Z = 26) und nicht 0 (Verschiebung bzw. Programm, A = 0). Wer O als 15 = `0000 1111` codiert, verschlüsselt ein anderes Zeichen.

ARCHIVWISSEN 2 · XOR – EXKLUSIVES ODER

a	b	a ⊕ b
0	0	0
0	1	1
1	0	1
1	1	0

XOR (Zeichen ⊕) verknüpft zwei Bits: kurz **gleich → 0, verschieden → 1**. Bitfolgen werden **Stelle für Stelle** verknüpft.

Beispiel: `0100 1111 ⊕ 1010 0110 = 1110 1001`.

Wichtige Regeln: a ⊕ 0 = a, a ⊕ a = 0 und deshalb

$$a \oplus k \oplus k = a$$

XOR ist **selbstinvers**: Wer mit k verschlüsselt, entschlüsselt mit derselben Operation und demselben k.

2 AUFTRAG ★ Gleich oder verschieden? EINZELARBEIT

Berechne beide Verknüpfungen. **Begründe** mit einer Regel aus Archivwissen 2, dass das zweite Ergebnis vorhersehbar war.

```
  1100 1010
⊕ 0110 0110
```

```
  1100 1010
⊕ 1100 1010
```


© Frank Juchim · Kryptos · für den Unterricht

= `1010 1100` = `0000 0000`

LÖSUNG

`1100 1010` ⊕ `0110 0110` = `1010 1100`; `1100 1010` ⊕ `1100 1010` = `0000 0000`, weil **a ⊕ a = 0** gilt – jedes Bit trifft auf ein gleiches Bit.

ARCHIVWISSEN 3 · VERNAM UND DAS ONE-TIME-PAD

Gilbert Vernam ließ 1917 eine Fernschreibmaschine jedes Zeichen der Nachricht mit einem Zeichen eines Schlüssel-Lochstreifens verknüpfen (damals im 5-Bit-Fernschreibcode, wir benutzen ASCII):

$$C = M \oplus K \qquad M = C \oplus K$$

Das Verfahren heißt **One-Time-Pad** (Einmalschlüssel), wenn der Schlüssel **(1) echt zufällig** erzeugt wird, **(2) mindestens so lang wie die Nachricht** ist, **(3) nur ein einziges Mal** benutzt wird und **(4) geheim** bleibt (nur Alice und Bob kennen ihn).

Dann ist es **perfekt sicher** (Claude Shannon, 1949): Zu einem Geheimtext passt **jeder** Klartext gleicher Länge mit genau einem Schlüssel, und alle Schlüssel sind gleich wahrscheinlich. Eve lernt aus dem Geheimtext nichts außer der Länge – auch mit beliebig viel Rechenzeit nicht. Der Preis ist das **Schlüsselproblem**: Ebenso viel geheimes Schlüsselmaterial wie Nachrichten muss vorher sicher übergeben werden.

ARCHIVWISSEN 4 · DER DOPPELTE SCHLÜSSEL

Benutzt jemand denselben Schlüssel K für zwei Nachrichten, gilt

$$C1 \oplus C2 = (M1 \oplus K) \oplus (M2 \oplus K) = \mathbf{M1 \oplus M2}$$

Der Schlüssel fällt heraus. Eve rät ein Wort (engl. *crib*), das in einer Nachricht vorkommen könnte, verknüpft es an jeder Position mit C1 ⊕ C2 und prüft, ob dort ein sinnvolles Stück der anderen Nachricht erscheint (**Crib-Dragging**). Das Byte `0000 0000` in C1 ⊕ C2 heißt: An dieser Stelle sind beide Klartextzeichen gleich. Genau so wurden ab den 1940er-Jahren Nachrichten entschlüsselt, deren Einmalschlüssel mehrfach benutzt worden waren (bekannt unter dem Namen Venona).

3 AUFTRAG ★★ Alice verschlüsselt BOJE PARTNERARBEIT

Alice verschlüsselt BOJE mit dem Einmalschlüssel `0110 1001 · 1111 0000 · 0011 0011 · 1010 0101`.

a) **Bestimme** den Geheimtext.

b) Bob entschlüsselt – **zeige** an einem Zeichen, dass er wieder BOJE erhält.

	B	O	J	E
Klartext M	`0100 0010`	`0100 1111`	`0100 1010`	`0100 0101`
Schlüssel K	`0110 1001`	`1111 0000`	`0011 0011`	`1010 0101`
C = M ⊕ K	**`0010 1011`**	**`1011 1111`**	**`0111 1001`**	**`1110 0000`**

LÖSUNG P3

a) Geheimtext `0010 1011 · 1011 1111 · 0111 1001 · 1110 0000`.

b) z. B. 1. Zeichen: `0010 1011` ⊕ `0110 1001` = `0100 0010` = 66 = **B** – XOR mit demselben Schlüsselbyte macht die Verschlüsselung rückgängig.

© Frank Juchim · Kryptos · für den Unterricht

4 AUFTRAG ★★ HALT oder LAUF? PARTNERARBEIT

Eve fängt den OTP-Geheimtext `0001 1110 · 1101 1010 · 0110 0111 · 1001 0001` ab.

a) **Bestimme** den Schlüssel, für den der Klartext HALT lautet, und den Schlüssel, für den er LAUF lautet.

b) **Erläutere** damit, was „perfekt sicher“ bedeutet.

	Zeichen 1	Zeichen 2	Zeichen 3	Zeichen 4
Geheimtext C	0001 1110	1101 1010	0110 0111	1001 0001
HALT	0100 1000	0100 0001	0100 1100	0101 0100
K_{HALT} = C ⊕ HALT	**0101 0110**	**1001 1011**	**0010 1011**	**1100 0101**
LAUF	0100 1100	0100 0001	0101 0101	0100 0110
K_{LAUF} = C ⊕ LAUF	**0101 0010**	**1001 1011**	**0011 0010**	**1101 0111**

LÖSUNG

a) K_{HALT} = `0101 0110 · 1001 1011 · 0010 1011 · 1100 0101`, K_{LAUF} = `0101 0010 · 1001 1011 · 0011 0010 · 1101 0111`.

b) Beide Schlüssel sind gleich wahrscheinlich (zufällig erzeugt). Der Geheimtext liefert also keinen Hinweis, welche Nachricht gesendet wurde; jeder Klartext der Länge 4 ist möglich. Auch Brute Force liefert nur alle Wörter der Länge 4 – ohne Entscheidung.

5 AUFTRAG ★★★ Der doppelte Schlüssel GRUPPENARBEIT

Zwei Funksprüche wurden mit **demselben** Einmalschlüssel verschlüsselt (Zeichenpositionen ab 0):

Position	0	1	2	3	4	5	6
C1	1001 0011	0111 0011	1110 1001	0000 0001	1011 0100	0101 0110	1100 0100
C2	1000 0100	0111 0111	1110 0100	0001 0010	1010 0111	0101 1011	1101 1000
D = C1 ⊕ C2	**0001 0111**	**0000 0100**	**0000 1101**	**0001 0011**	**0001 0011**	**0000 1101**	**0001 1100**

a) **Bestimme** D = C1 ⊕ C2.

b) Eve vermutet in einer Nachricht das Wort UHR (U = `0101 0101`, H = `0100 1000`, R = `0101 0010`). Verknüpft UHR mit D an den Positionen 0 bis 4 (**eine Position je Person**) und notiert die Zeichen.

c) **Gib** beide Nachrichten **an**.

UHR ab Position	Bits (UHR ⊕ D)	Zeichen	sinnvoll?
0	0100 0010 · 0100 1100 · 0101 1111	B L _	kaum
1	0101 0001 · 0100 0101 · 0100 0001	Q E A	nein
2	0101 1000 · 0101 1011 · 0100 0001	X [A	nein
3	0100 0110 · 0101 1011 · 0101 1111	F [_	nein
4	0100 0110 · 0100 0101 · 0100 1110	**F E N**	**ja**

LÖSUNG P1 P8

a) D = `0001 0111 · 0000 0100 · 0000 1101 · 0001 0011 · 0001 0011 · 0000 1101 · 0001 1100`.

b) Tabelle oben: Nur Position 4 liefert ein sinnvolles Wortstück **FEN** (Positionen 2 und 3 enthalten sogar Nicht-Buchstaben).

c) Eine Nachricht endet auf UHR, die andere auf FEN; Eve rät weiter (z. B. HAFEN) und erhält **VIERUHR** und **AMHAFEN**.

Typische Fehler: Positionen ab 1 zählen (P1); D mit dem Schlüssel verwechseln.

© Frank Juchim · Kryptos · für den Unterricht

6 AUFTRAG ★★ Sparversion EINZELARBEIT

Ein Fernschreibamt spart Schlüssel und verknüpft jedes Zeichen mit demselben Byte k = `0010 0001`.

a) **Verschlüssele** EBBE (als Bits).

b) **Entscheide begründet**, ob dieses Verfahren mono- oder polyalphabetisch ist, und **erläutere** einen Angriff.

k = 0010 0001	E	B	B	E
Klartext	0100 0101	0100 0010	0100 0010	0100 0101
Geheimtext	**0110 0100**	**0110 0011**	**0110 0011**	**0110 0100**

LÖSUNG P5 P7 P8

a) EBBE → `0110 0100 · 0110 0011 · 0110 0011 · 0110 0100` (als Zeichen „dccd“).

b) **Monoalphabetisch:** Gleiche Klartextzeichen (B, B) ergeben gleiche Geheimtextbytes. Angriff: Häufigkeitsanalyse der Bytes (häufigstes Byte ≙ E bzw. Leerzeichen) oder Brute Force über nur 256 Schlüssel.

Merke (P8): Auch zweimal XOR mit k_1 und k_2 hilft nicht: $(M \oplus k_1) \oplus k_2 = M \oplus (k_1 \oplus k_2)$ wirkt wie **ein** Schlüssel.

© Frank Juchim · Kryptos · für den Unterricht

ABI-CHECK · ca. 15 min 11 BE

a) **Bestimmen** Sie die Bitfolge des Geheimtextes, der entsteht, wenn das Wort OST (ASCII) mit dem Schlüssel `1010 0110 0101 1100 1111 0000` durch XOR verschlüsselt wird. **[3 BE]**

ERWARTUNG a) P1

O, S, T = `0100 1111 0101 0011 0101 0100`; Geheimtext **`1110 1001 0000 1111 1010 0100`**. [ASCII korrekt 1, XOR korrekt 2]

b) **Erläutern** Sie, unter welchen Bedingungen an den Schlüssel die XOR-Verschlüsselung perfekt sicher ist. **[3 BE]**

ERWARTUNG b)

Schlüssel echt zufällig, mindestens so lang wie die Nachricht, nur einmal verwendet, geheim; dann passt zu einem Geheimtext jeder Klartext gleicher Länge (mit genau einem, gleich wahrscheinlichen Schlüssel). [je Bedingung 0,5; Erläuterung 1]

c) „Wird derselbe Einmalschlüssel für zwei Nachrichten verwendet, kann ein Angreifer Informationen über die Klartexte gewinnen, ohne den Schlüssel zu kennen."
Entscheiden Sie **begründet**, ob diese Aussage korrekt ist. **[3 BE]**

ERWARTUNG c) P8

Korrekt [1]: C1 ⊕ C2 = M1 ⊕ M2, der Schlüssel fällt heraus; aus Nullbytes erkennt man gleiche Zeichen, mit vermuteten Wörtern (Crib) lassen sich Teile beider Klartexte gewinnen [2].

d) **Begründen** Sie, dass eine XOR-Verschlüsselung, bei der jedes Zeichen mit demselben 8-Bit-Schlüssel verknüpft wird, mit einer Häufigkeitsanalyse angreifbar ist. **[2 BE]**

ERWARTUNG d) P5 P10

Gleiche Zeichen → gleiche Bytes (monoalphabetisch) [1]; die Verteilung der Bytes entspricht der Buchstabenverteilung der Sprache, das häufigste Byte steht für E (bzw. Leerzeichen) [1].
Typischer Fehler P10: nur „weil der Schlüssel kurz ist".

IM LAB ÜBEN Level 3.1

Level 3.1 „Vernams Lochstreifen" im Kryptolabor mit der **Bit-Lupe** (Zeichen ↔ Bits, XOR, Doppelnutzungs-Modus mit Crib-Schieber). Trainingsplatz: „XOR-Bytes" und „Einmalschlüssel doppelt".

© Frank Juchim · Kryptos · für den Unterricht